

DA6401 — Assignment 3: Transformer for Neural Machine Translation

Implementation of the Transformer architecture from scratch in PyTorch, trained on the Multi30k German→English dataset.

W&B Report: <https://wandb.ai/ashishmeshram229-indian-institute-of-technology-madras/da6401-assignment-3/reports/DA6401-Assignment-3-Transformer-NMT-Experiments-Analysis--VmlldzoxNjk3MTIxNg?accessToken=50wmrc2383rwrnk10na4x8hyzsl9euiurtzmljg3mt5z78y7e5ehn6ncvbqqdcj2>

Overview

This assignment implements the landmark architecture from "*Attention Is All You Need*" (Vaswani et al., 2017) without using any high-level wrappers like `nn.Transformer` or `nn.MultiheadAttention`. The model is trained end-to-end as a Neural Machine Translation (NMT) system translating German to English.

Dataset: [Multi30k](#) — 29,000 training pairs, 1,014 validation pairs, 1,000 test pairs.

Model Architecture

Hyperparameter	Value
<code>d_model</code>	256
<code>num_heads</code>	8
<code>num_layers</code>	4 (encoder + decoder)
<code>d_ff</code>	1024
<code>dropout</code>	0.1
<code>max_len</code>	100
<code>max_vocab_size</code>	12,000
<code>positional_encoding</code>	sinusoidal
<code>label_smoothing</code>	0.1

All sub-layers follow the original paper: scaled dot-product attention, multi-head attention with 8 parallel heads, position-wise feed-forward networks with ReLU, Add & Norm residuals, and sinusoidal positional encodings.

Project Structure

```

da6401_assignment_3/
├── model/
│   ├── attention.py          # Scaled dot-product + MultiHeadAttention
│   ├── encoder.py           # Encoder layer + Encoder stack
│   ├── decoder.py           # Decoder layer + Decoder stack
│   ├── positional_encoding.py # Sinusoidal and learned PE
│   ├── feed_forward.py      # Point-wise FFN
│   └── transformer.py       # Full Seq2Seq Transformer
├── training/
│   ├── train.py             # Training loop, DEFAULT_CONFIG
│   ├── scheduler.py         # Noam LR scheduler
│   └── loss.py              # Label smoothing cross-entropy
├── data/
│   └── dataset.py           # Multi30k loader, tokenisation (spaCy)
├── inference/
│   └── greedy_decode.py     # Token-by-token greedy decoding
├── utils/
│   ├── masking.py          # Padding mask + causal look-ahead mask
│   └── metrics.py          # BLEU evaluation, attention logging
├── colab_wandb_train.py     # Colab entry-point, all experiment
├── configs
│   ├── fetch_runs.py       # W&B run data fetcher
│   ├── create_wandb_report.py # Programmatic W&B report generator
│   └── README.md

```

Implementation Details

Task 1 — Scaled Dot-Product & Multi-Head Attention

Attention is implemented as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^{\top}}{\sqrt{d_k}}\right)V$$

Multi-head attention projects Q, K, V into `num_heads` subspaces, computes attention in parallel, and concatenates the results. Both padding masks (for encoder/decoder) and causal look-ahead masks (for the decoder) are implemented. `torch.nn.MultiheadAttention` is not used.

Task 2 — Encoder and Decoder Stacks

Sinusoidal positional encoding:

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right), \quad PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

Post-LayerNorm is used (as in the original paper). The point-wise FFN is:

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

Task 3 — Training Pipeline

- **Label Smoothing:** $\epsilon = 0.1$, implemented as a custom loss that redistributes probability mass uniformly.
- **Noam Scheduler:**

$$\text{lr} = d_{\text{model}}^{-0.5} \cdot \min(\text{step}^{-0.5}, \text{step}^{-1.5})$$

- **Greedy Decoding:** Inference generates tokens one at a time using argmax at each step until `<eos>` or `decode_max_len=80`.

Training

Setup

```
git clone https://github.com/MiRL-IITM/da6401_assignment_3
cd da6401_assignment_3
pip install torch datasets spacy wandb evaluate
python -m spacy download en_core_web_sm
python -m spacy download de_core_news_sm
```

Run all experiments (Colab)

```
# Fast profile (4 epochs, for testing)
python colab_wandb_train.py --profile fast --entity YOUR_WANDB_ENTITY

# Full experiment suite (10 epochs, all 9 runs)
python colab_wandb_train.py --profile pro --entity YOUR_WANDB_ENTITY

# Run a specific experiment group only
python colab_wandb_train.py --profile pro --only noam_vs_fixed_lr --
entity YOUR_WANDB_ENTITY
```

Run a single training

```
from train import DEFAULT_CONFIG, run_training
from copy import deepcopy

config = deepcopy(DEFAULT_CONFIG)
config.update({
    "epochs": 10,
    "use_noam": True,
    "use_scaling": True,
    "label_smoothing": 0.1,
    "positional_encoding": "sinusoidal",
    "use_wandb": False,
```

```
}  
run_training(config)
```

Experiments & Results

All 9 runs are logged to W&B. Full analysis is in the report linked at the top.

Section 2.1 — Noam Scheduler vs Fixed LR

Run	Test BLEU	Val Loss	Val Token Acc
<code>noam_scheduler</code>	35.76	2.837	63.5%
<code>fixed_lr_1e-4</code>	30.21	3.222	57.8%

The Noam scheduler improves test BLEU by **+5.55 points** over a fixed LR. The warmup phase stabilises early training when attention weights are randomly initialised.

Section 2.2 — Scaling Factor $1/\sqrt{dk}$ Ablation

Run	Test BLEU	Final Q Grad Norm	Final K Grad Norm
<code>with_sqrt_dk_scaling</code>	35.76	0.760	0.780
<code>without_sqrt_dk_scaling</code>	34.13	2.853	2.901

Without scaling, Q/K gradient norms are **3.75x higher**, indicating the softmax operates in its saturated region where gradients are near zero during learning, leading to unstable training.

Section 2.3 — Attention Rollout & Head Specialization

Attention weights extracted from the last encoder layer at step 4540. Key observations from the 8 head heatmaps:

- **Head 4** — near-exclusive self-attention (identity / residual pass-through head)
- **Head 3** — sharp column pattern attending to the subject noun from all positions
- **Head 1** — local block structure, short-range syntactic dependency head
- **Head 6** — long-range attention to sentence-boundary positions
- **Heads 2, 7** — near-uniform flat distributions (redundant heads)

Heads 2 and 7 in the deepest layer show near-zero selectivity, consistent with attention head redundancy documented in the literature.

Section 2.4 — Sinusoidal PE vs Learned Embeddings

Run	Test BLEU	Val BLEU
<code>sinusoidal_position</code>	35.76	35.56

Run	Test BLEU	Val BLEU
<code>learned_position</code>	35.82	35.19

Results are nearly identical on Multi30k (short sentences, max ~30 tokens). The theoretical advantage of sinusoidal PE — extrapolation to unseen sequence lengths via fixed periodic functions — is not observable on this dataset but would matter on longer corpora.

Section 2.5 — Label Smoothing

Run	Test BLEU	Train Confidence	Val Confidence	Train Loss
<code>label_smoothing_0_1</code> ($\epsilon=0.1$)	35.76	0.450	0.497	2.858
<code>label_smoothing_0_0</code> ($\epsilon=0.0$)	36.35	0.500	0.542	1.869

As expected, $\epsilon=0.1$ produces lower prediction confidence (capped by the soft target) and higher training loss (since the objective minimum is non-zero by construction). On Multi30k $\epsilon=0.0$ achieves slightly higher BLEU; label smoothing's regularisation benefit is more pronounced on larger, noisier datasets with longer training.

W&B Experiment Groups

Group	Runs
<code>noam_vs_fixed_lr</code>	<code>noam_scheduler</code> , <code>fixed_lr_1e_4</code>
<code>scaling_factor_ablation</code>	<code>with_sqrt_dk_scaling</code> , <code>without_sqrt_dk_scaling</code>
<code>attention_visualization</code>	<code>attention_head_analysis</code>
<code>positional_encoding</code>	<code>sinusoidal_position</code> , <code>learned_position</code>
<code>label_smoothing</code>	<code>label_smoothing_0_1</code> , <code>label_smoothing_0_0</code>

Requirements

```
torch
numpy
spacy
wandb
datasets
evaluate
tqdm
```

References

- Vaswani, A. et al. (2017). *Attention Is All You Need*. NeurIPS.
https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf
- Voita, E. et al. (2019). *Analyzing Multi-Head Self-Attention: Specialized Heads Do the Heavy Lifting, the Rest Can Be Pruned*. ACL.
- Michel, P. et al. (2019). *Are Sixteen Heads Really Better than One?* NeurIPS.
- Multi30k dataset: <https://huggingface.co/datasets/bentrevett/multi30k>